

Stake2Care ERC4626 Impact Vault audit Report

This audit report provides a comprehensive analysis of the Stake2Care ERC4626 Impact Vault protocol, conducted by [HHK](#), focusing on identifying potential security vulnerabilities, gas optimizations and assessing overall code quality.

Table of Contents

- [Project Presentation](#)
- [Auditor presentation](#)
- [Scope](#)
- [Findings Severity](#)
- [Findings](#)
- [Critical Findings](#)
- [High Findings](#)
- [Medium Findings](#)
 - [1. Medium - Unlimited timelock reset DoS via `minimalTransfer` parameter](#)
 - [Technical Details](#)
 - [Impact](#)
 - [Recommendation](#)
 - [Developer Response](#)
- [Low Findings](#)
 - [1. Low - Unable to claim Merkl rewards](#)
 - [Technical Details](#)
 - [Impact](#)
 - [Recommendation](#)
 - [Developer Response](#)
- [Gas Saving Findings](#)
 - [1. Gas - `maxRedeem\(\)` can be simplified](#)
 - [Technical Details](#)
 - [Recommendation](#)
 - [Developer Response](#)
- [Informational Findings](#)
 - [1. Informational - Preview functions use potentially stale data](#)
 - [Technical Details](#)
 - [Impact](#)
 - [Recommendation](#)
 - [Developer Response](#)
 - [2. Informational - Deployment scripts missing vault seeding](#)
 - [Technical Details](#)
 - [Impact](#)
 - [Recommendation](#)
 - [Developer Response](#)
- [Conclusion](#)

Project Presentation

Stake2Care ERC4626 Impact Vault

Stake2Care ERC4626 Impact Vault extends the original Stake2Care protocol by enabling deposits into Morpho and other ERC4626-compliant vaults. Users deposit assets while retaining their principal, and all generated yield is automatically donated to charity (Doctors Without Borders).

Auditor presentation

HHK is a freelance smart contract developer and security researcher with a strong track record. Multiple top 10 during audit contests and has conducted over 25 audits with [yAudit](#). Specializing in smart contract security since early 2023, HHK's audit portfolio is available [here](#). For inquiries, please contact hhk.contact@proton.me.

Scope

The scope of the review consisted of the following contracts at the specific commit:

```
ERC4626ImpactVault.sol
```

The contracts of the Stake2Care ERC4626 Impact Vault [Repo](#) were reviewed over 2 days. The repository was under active development during the review, but the review was limited to the latest commit at the start of the review. This was commit [f751f29d894963f168cd91b608a96c372cbe7380](#) for the Stake2Care ERC4626 Impact Vault repo.

This review is a code review to identify potential vulnerabilities in the code. The reviewers did not investigate security practices or operational security and assumed that privileged accounts could be trusted. The reviewers did not evaluate the security of the code relative to a standard or specification. The review may not have identified all potential attack vectors or areas of vulnerability. By deploying or using the code, Stake2Care and users of the contracts agree to use the code at their own risk.

Findings Severity

Impact/Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Informational

Suggestions to enhance performance (gas consumption) and architecture are provided as gas and informational findings.

Note that the assessment of findings severity is subjective and may vary depending on individual perspectives, system configurations, and specific use cases. What one user considers a critical issue may be deemed less severe by another based on different criteria and priorities.

Findings

Critical Findings

None.

High Findings

None.

Medium Findings

1. Medium - Unlimited timelock reset DoS via `minimalTransfer` parameter

An attacker can permanently block donation collection by calling `collectDonations()` with a crafted parameter that resets the 3-day timelock without transferring any funds.

Technical Details

The `collectDonations()` function allows anyone to pass a custom `minimalTransfer` parameter that overrides the stored `minimalCollectAmount` threshold.

The function has two key checks:

1. [Line 553](#): `redeemableAssets > totalSupply_ + minimalTransfer` - gates entry to the timelock logic
2. [Line 564](#): `collectedAmount > minimalTransfer` - gates actual donation transfer

However, the timelock reset at [lines 574-580](#) occurs regardless of whether the second check passes.

An attacker can craft a `minimalTransfer` value that:

1. Is low enough to pass the first check
2. Is high enough to fail the second check

This causes the 3-day timelock to reset without any donations being transferred.

Attack scenario:

- Surplus was locked 3 days ago at 100 tokens (`timeLockedSurplus.surplus = 100`)
- Current surplus has grown to 101 tokens due to yield
- Attacker calls `collectDonations(100)` just after the timelock expires
- First check passes: `101 > totalSupply + 100` ✓
- `collectedAmount = min(101, 100) = 100`
- Second check fails: `100 > 100` ✗
- Timelock resets to `block.timestamp + 3 days` without any transfer

The attacker can repeat this every 3 days indefinitely, permanently blocking donations.

Impact

Medium. An attacker can permanently prevent yield donations from reaching the charity by repeatedly resetting the timelock. The attack only costs gas fees and can be executed by anyone.

Recommendation

Only reset the timelock when donations are actually collected. Move the timelock update inside the successful collection block. Alternatively, remove the `minimalTransfer` parameter entirely and always use the stored `minimalCollectAmount` threshold.

Developer Response

Partially Acknowledged

- Possible scenario is attacker specifying `minimalTransfer = Timelocked Surplus + 1` while collection window is opened (Timelock elapsed)
- Block is not permanent: Attack is only possible after the timelock has elapsed and before surplus is collected (see [Line 556](#)) To permanently block donations, attacker would have to consistently outbid Team calls to `collectDonation` in the block, when timelock elapses. In the limit case, attacker would have to pay the full surplus to be donated in priority fees/ Builder bribe every three days, to maintain the lock, which is disproportionately costly.

Implemented fix: We upper bound the `minimalTransfer` param by the stored `minimalCollectAmount` threshold. Attacker cannot reset timelock once surplus is above the stored threshold.

Fixed in commit [d9a3394](#).

Low Findings

1. Low - Unable to claim Merkl rewards

The ImpactVault cannot claim MORPHO rewards distributed via Merkl, resulting in permanent loss of entitled rewards.

Technical Details

MetaMorpho vaults like Steakhouse USDC distribute MORPHO rewards to depositors via the [Merkl distributor](#). As a depositor in MetaMorpho, the ImpactVault is entitled to these rewards.

However, Merkl requires the recipient to either call `claim()` directly or pre-approve an operator via `toggleOperator()`. Since the ImpactVault has no function to interact with Merkl, the MORPHO rewards will remain unclaimed forever.

Impact

Low. The vault loses MORPHO token rewards it is entitled to as a depositor.

Recommendation

Add an owner only function to claim Merkl rewards or call `toggleOperator()` on the Merkl distributor during deployment to pre-approve the owner as an operator.

Developer Response

Partially Acknowledged

- Current contract unable to claim for himself, and doesn't toggle Merkl operator..
- No permanent loss of funds as Morpho documentation recommends to open a ticket with Merkl, who can toggle operator on behalf of contract.

Implemented fix: Created `MetaMorphoImpactVault` inheriting `ERC4626ImpactVault`, adding an owner-only `toggleMerklOperator(operator)` function that calls `Merkl.toggleOperator(address(this), operator)` on an immutable Merkl distributor set at construction.

Fixed in commit [11b297b](#).

Gas Saving Findings

1. Gas - `maxRedeem()` can be simplified

Technical Details

The current implementation:

```
function maxRedeem(address owner) public view override(ERC4626) returns
(uint256) {
    uint256 maxAssets = maxWithdraw(owner);
    uint256 neededShares = previewWithdraw(maxAssets);
    uint256 ownerShares = balanceOf(owner);
    return neededShares < ownerShares ? neededShares : ownerShares;
}
```

`maxWithdraw` already guarantees that `previewWithdraw(maxAssets) <= balanceOf(owner)` through its binary search logic. The `balanceOf` call and `min` comparison are therefore redundant.

Recommendation

Simplify to:

```
function maxRedeem(address owner) public view override(ERC4626) returns
(uint256) {
    return previewWithdraw(maxWithdraw(owner));
}
```

Developer Response

Fixed in commit [d9a3394](#).

Informational Findings

1. Informational - Preview functions use potentially stale data

Technical Details

The original preview functions (e.g., `_previewRedeemGivenGlobals`) computed share/asset conversions using:

```
uint256 totalAssets_ = Math.mulDiv(
    g.ourTotalPosition,
    g.underlyingVaultTotalAssets,
    g.underlyingVaultTotalSupply,
    Math.Rounding.Down
);
```

On MetaMorpho vaults, `totalSupply()` doesn't reflect accrued performance fees until they are minted during a state-changing operation. However, `convertToAssets()` and `convertToShares()` internally account for these pending fees. This mismatch could lead to small pricing inconsistencies between preview calls and actual execution.

Impact

Informational.

Recommendation

Use `underlyingVault.convertToAssets()` and `convertToShares()` instead of manual `mulDiv` calculations with `totalAssets/totalSupply` ratios.

Developer Response

Fixed in commit `466bc4d`.

2. Informational - Deployment scripts missing vault seeding

Technical Details

The constructor comment at [line 89](#) recommends a "donation" but a direct asset transfer would be captured by `collectDonations()` and sent to the owner. A **deposit** is required to properly seed the vault.

The deployment scripts do not perform this seeding.

Impact

Informational. Without seeding, the vault may experience rounding edge cases on initial deposits.

Recommendation

Update the constructor comment to say "deposit" instead of "donation", and add a seeding deposit of `MIN_DEPOSIT` to the deployment scripts.

Developer Response

Fixed in commit [d9a3394](#).

Conclusion

The Stake2Care ERC4626 Impact Vault extends the protocol's capabilities by enabling integration with Morpho and other ERC4626-compliant vaults. The codebase is well-structured and demonstrates careful consideration of edge cases. While concerns were raised regarding the complexity of certain functions such as `maxWithdraw()`, which uses binary search logic, this approach appears necessary to ensure proper integration with any ERC4626 vault, particularly those implementing fees. All identified issues were promptly addressed by the development team.